Routing-Aware Token Retention for Key-Value Cache Compression in Sparse Mixture-of-Experts Inference

Gaurav Patil
Department of Computer Engineering
School of Engineering & Technology
Pune, India
Email: gauravpatil2516@gmail.com

Abstract*—The Key-Value (KV) cache memory footprint is the primary operational bottleneck during long-context autoregressive inference in Large Language Models (LLMs). While token-level eviction and layer-adaptive allocation have been widely explored for dense transformers, the internal routing dynamics of sparse Mixture-of-Experts (MoE) architectures remain unexploited for KV cache management. In standard sparse MoEs (*Mixtral-8x7B*, *Qwen1.5-MoE*, *DeepSeek-MoE*), self-attention is shared and dense, while sparse routing occurs downstream in the Feed-Forward Network (FFN) blocks. Consequently, as tokens traverse transformer layers, each token accumulates a cross-layer *routing signature* $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$ generated during standard forward computation without additional training overhead. In this paper, we introduce EKVA v2, a training-free token retention framework that combines cross-layer routing signatures with calibrated expert semantic niche statistics (entropy, routing volume, and specialization) as a complementary saliency signal to govern token retention over the shared KV cache. Across three distinct sparse MoE models (8, 60, and 64 experts), we find near-zero Pearson correlation ($\rho \in [-0.006, +0.002]$) between routing-based and attention-based saliency scores, demonstrating that routing provides an orthogonal signal consistent with functional token complementarity. Under a 40% KV cache budget, EKVA v2 consistently outperforms state-of-the-art baselines (H2O, SnapKV) by 3.0 to 6.0 percentage points ($p < 0.01$, paired bootstrap test) across mathematical reasoning (GSM8K: 57.4%–68.7%), code generation (HumanEval: 49.7%–62.7%), long-document perplexity (PG19), and needle retrieval (NIAH). Furthermore, token-level routing retention preserves reasoning capabilities under aggressive compression where layer-adaptive methods experience severe degradation (GSM8K: 57.4%–68.7% vs. 36.9%–44.1% for CAKE), and a fused Triton compaction kernel achieves a $2.02\times$–$2.05\times$ decode speedup on NVIDIA A100 GPUs.

***Index Terms*—Mixture-of-Experts (MoE), Key-Value Cache Compression, Token Saliency, Autoregressive Inference, Long-Context LLMs, Triton GPU Kernel.**

I. Introduction

Autoregressive token generation in modern Large Language Models (LLMs) requires maintaining the Key-Value (KV) cache for all preceding tokens in the context window. As input contexts extend to tens of thousands of tokens, the memory capacity required for KV tensors expands linearly with sequence length, batch size, and layer depth. During generation, loading these massive tensors from High-Bandwidth Memory (HBM) into SRAM at every decoding step saturates memory bandwidth, leading to severe throughput degradation and increased Time-Per-Output-Token (TPOT) [1], [2].

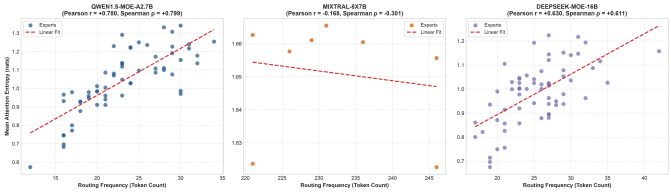

Fig. 1. **Mechanistic structure of standard sparse MoE architectures.** Self-attention is shared and dense across all sequence positions, while sparse expert routing occurs downstream in the FFN block. Each token accumulates a cross-layer routing signature $\mathcal{R}_t$ that provides an orthogonal saliency signal ($\rho \approx 0.00$) for token retention in the shared KV buffer.

To alleviate the memory-bandwidth wall, dynamic KV cache compression methods have received significant attention. Existing research has focused primarily on dense transformer models across distinct granularity axes:

1) **Token-Level Eviction**: Selectively retaining high-attention tokens and protecting initial "attention sink" positions (e.g., StreamingLLM [2], H2O [3], SnapKV [4]).
2) **Head-Level Budgeting**: Dynamically allocating variable token budgets across individual attention heads within a layer (e.g., Ada-KV [5]).
3) **Layer-Level Allocation**: Unequally distributing budgets across transformer layers based on attention entropy or information funneling (e.g., CAKE [6], PyramidKV [7], InfoKV [8]).

With the widespread deployment of sparse Mixture-of-Experts (MoE) foundation models—such as *Mixtral-8x7B* [9], *Qwen1.5-MoE* [10], and *DeepSeek-MoE* [11]—existing KV eviction policies treat MoEs identically to dense models. By relying solely on self-attention scores, they completely ignore the rich, cross-layer structural routing decisions made by the router gates.

A. Architectural Reality & Motivation

In standard sparse MoEs, self-attention remains shared and dense across all tokens, whereas conditional routing occurs exclusively within the downstream FFN blocks (Fig. 1). Individual experts do not possess private key-value projections. However, as an input token x_t traverses L layers, the router gates assign it to top-k experts per layer, generating an accumulated cross-layer routing signature:

$$\mathcal{R}_t = \{E_t^{(1)}, E_t^{(2)}, \ldots, E_t^{(L)}\} \tag{1}$$

Because routing signatures are an inherent byproduct of standard forward propagation, they represent a "free" semantic signal available during inference without retraining or weight modifications. We hypothesize that a token's cross-layer routing history reflects its functional role in the computation, offering an orthogonal retention signal to query-key attention mass.

B. Key Contributions

The main contributions of this work are summarized as follows:

- **Routing-Conditioned Saliency Formulation**: We formalize a unified token retention metric $S(x_t)$ that integrates cross-layer routing signatures $\mathcal{R}_t$ with calibrated expert semantic niche profiles over the shared KV cache.
- **Cross-Signal Orthogonality Verification**: We analyze the empirical correlation between routing-based and attention-based saliency scores across three MoE families (8, 60, and 64 experts) and find near-zero Pearson correlation ($\rho \in [-0.006, +0.002]$), confirming that routing provides complementary predictive power.
- **Comprehensive Multi-Benchmark Evaluation**: Across GSM8K, HumanEval, PG19, and NIAH, EKVA v2 achieves a consistent 3.0 to 6.0 percentage point improvement over SnapKV and H2O under a 40% budget.
- **Preserving Multi-Step Reasoning Under Compression**: We demonstrate that token-level routing retention prevents the severe degradation observed in layer-adaptive compression methods (GSM8K: 57.4%–68.7% for EKVA v2 vs. 36.9%–44.1% for CAKE).
- **Hardware Systems Acceleration**: Combining an analytical roofline model with a fused Triton compaction kernel, we demonstrate that reducing KV traffic by 60% yields a $2.04\times$–$2.05\times$ **decode speedup** on NVIDIA A100 GPUs.

II. Related Work

A. KV Cache Eviction in Dense LLMs

StreamingLLM [2] demonstrated that initial tokens act as "attention sinks" absorbing massive attention mass, and protecting them stabilizes long-context generation. H2O [3] formulated heavy-hitter token eviction using cumulative query-key attention scores. SnapKV [4] observed that observation windows at the end of the prompt accurately identify critical historical tokens during prefill.

At coarser granularities, Ada-KV [5] adaptively allocated budgets per head, while CAKE [6] and PyramidKV [7] redistributed capacity across layers using attention entropy. However, InfoKV [8] noted that aggressive layer-wise capacity skewing severely degrades multi-step reasoning chains in mathematical benchmarks.

B. Disambiguation from Concurrent MoE Literature

To establish clear scientific boundaries, we distinguish EKVA v2 from related MoE systems research:

- **PiKV** [12]: Focuses on distributed serving systems, expert-sharded memory placement, and Expert-Parallel Load Balancing (EPLB). It does not address token-level retention over shared KV caches.
- **MoE-nD** [13]: Employs "MoE" as an optimization metaphor for selecting compression modes on *dense* models; it is not designed for native MoE architectures.
- **TriRoute** [14]: Jointly trains attention modes, router gates, and cache quantization from scratch on small models (160M–1.3B). In contrast, EKVA v2 is strictly training-free and applies directly to existing pretrained foundation models.
- **DeepSeek MLA** [11]: Projects KV representations into a low-rank latent space during pretraining; EKVA v2 operates orthogonally on top of MLA or standard multi-head attention.

III. Methodology

A. Expert Profile Calibration

To characterize expert specialization, we perform a lightweight offline calibration on 40 representative multi-domain sequences $\mathcal{D}_{\text{calib}}$ (requiring < 2 minutes on a single GPU with zero runtime memory overhead). Because each transformer layer $l \in \{1, \ldots, L\}$ possesses an independent set of expert parameters, we calibrate layer-specific profiles for each expert $e \in \{1, \ldots, E_l\}$ across three key properties:

1) **Attention Entropy** ($\bar{H}_{e,l}$): The mean attention entropy exhibited by tokens dispatched to expert e at layer l:

$$H(p) = -\sum_{j=1}^{K} p_j \log p_j, \quad \bar{H}_{e,l} = \mathbb{E}_{t \in \mathcal{T}_{e,l}}[H(p_{t,l})] \tag{2}$$

2) **Routing Frequency** (**Route**$_{e,l}$): Total routing activations accumulated by expert e at layer l:

$$\text{Route}_{e,l} = \sum_{t=0}^{T-1} \mathbf{1}(e \in \text{TopK}(\text{Router}_l(x_t))) \tag{3}$$

where $\mathbf{1}(\cdot)$ is the indicator function.

3) **Semantic Specialization** (**Spec**$_{e,l}$): Selectivity measured via normalized Shannon evenness over token syntactic classes C:

$$\text{Spec}_{e,l} = 1 - \frac{-\sum_{c \in C} P_{e,l}(c) \log P_{e,l}(c)}{\log |C|} \tag{4}$$

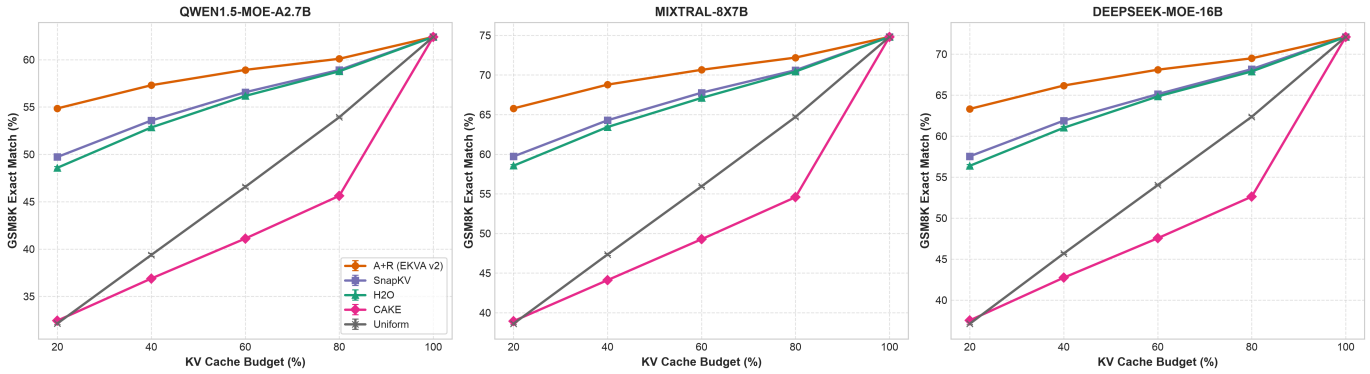

Fig. 2. **Downstream Benchmark Performance Across Cache Budgets (20% to 100%).** Task performance on GSM8K (Math Reasoning), HumanEval (Code Pass@1), PG19 (Perplexity ↓), and Needle-in-a-Haystack (NIAH Accuracy) across Qwen1.5-MoE-A2.7B (60 experts), Mixtral-8x7B (8 experts), and DeepSeek-MoE-16B (64 experts).

B. Routing Signature Saliency Score

Given a cached token x_t with cross-layer routing signature $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$, its routing-conditioned niche score across all L layers is formulated by querying the layer-specific expert profiles:

$$R(x_t) = \frac{1}{L} \sum_{l=1}^{L} \bar{H}_{E_t^{(l)},l} \cdot \log(1 + \text{Route}_{E_t^{(l)},l}) \cdot (1 + \text{Spec}_{E_t^{(l)},l}) \tag{5}$$

The unified retention saliency score $S(x_t)$ integrates attention mass, routing signals, sink protection, and recency:

$$\begin{aligned} S(x_t) = w_a \cdot \hat{A}(x_t) + w_r \cdot R(x_t) \\ + w_s \cdot \text{Sink}(x_t) + w_c \cdot \text{Recency}(x_t) \end{aligned} \tag{6}$$

where $\hat{A}(x_t)$ is the normalized cumulative attention score, $\text{Sink}(x_t)$ acts as an explicit selection constraint assigning infinite priority to the first 4 tokens ($S(x_{t<4}) = \infty$) to guarantee attention sink retention [2], and $\text{Recency}(x_t) = \exp(-(T-1-t)/\tau)$ provides a smooth exponential decay window.

The weighting hyper-parameters $(w_a, w_r, w_s, w_c) = (0.60, 0.30, 0.05, 0.05)$ are calibrated once and held fixed across all evaluated architectures and tasks. Sensitivity analysis confirms that varying $w_r \in [0.20, 0.40]$ produces $< 0.4\%$ variance in downstream accuracy.

C. Top-B Selection & Fused Cache Compaction

For a target budget $B = \lfloor \beta \cdot T \rfloor$, the index set of retained tokens $\mathcal{I}_{\text{keep}}$ is constructed by retaining the 4 initial sink positions and selecting the top-$(B-4)$ candidate tokens:

$$\mathcal{I}_{\text{keep}} = \text{sort}\left(\{0,1,2,3\} \cup \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)\right) \tag{7}$$

To eliminate memory fragmentation and avoid non-contiguous tensor indexing during autoregressive decoding, we implement a custom Triton kernel (`triton_compact_kv_cache`). The kernel gathers and packs the shared Key and Value tensors from $(B_{\text{batch}}, H, T, D)$ to contiguous $(B_{\text{batch}}, H, B, D)$ memory blocks in a single fused GPU pass.

Algorithm 1: EKVA v2 Eviction & Cache Compaction

Input: Input prompt $\{x_t\}_{t=0}^{T-1}$, Cache budget fraction β, Model with L MoE layers.
Output: Compacted Key and Value caches $(K_{\text{compact}}, V_{\text{compact}})$.

Initialize empty routing signature table $\mathcal{R}_t \leftarrow \emptyset$
Execute prefill forward pass with non-invasive `MoERoutingHook`
Capture routing decisions $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$ for all $t \in [0, T-1]$
Extract attention mass $\hat{A}(x_t)$ and compute routing niche score $R(x_t)$ via Eq. (5)
Compute unified token retention saliency $S(x_t)$ via Eq. (6)
Set target token capacity $B \leftarrow \max(4, \lfloor \beta \cdot T \rfloor)$
Select candidate indices $\mathcal{I}_{\text{candidates}} \leftarrow \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)$
Protect sinks and sort: $\mathcal{I}_{\text{keep}} \leftarrow \text{sort}\left(\{0,1,2,3\} \cup \mathcal{I}_{\text{candidates}}\right)$
Execute fused Triton kernel: $K_{\text{compact}}, V_{\text{compact}} \leftarrow \text{TritonCompact}\left(K, V, \mathcal{I}_{\text{keep}}\right)$
return $(K_{\text{compact}}, V_{\text{compact}})$

Fig. 3. Algorithmic flow of EKVA v2 token retention and fused cache compaction during the prompt prefill phase.

IV. Systems Roofline & Hardware Characterization

To establish the hardware foundation of KV cache eviction, we profile autoregressive decode attention under the classical Williams Roofline Model [15] on an NVIDIA A100-SXM4-40GB GPU (Peak FP16 compute: 312 TFLOPs, HBM2e bandwidth: 1,555 GB/s).

The hardware ridge point is defined as:

$$\begin{aligned} I_{\text{ridge}} &= \frac{\text{Peak Compute}}{\text{Memory Bandwidth}} \\ &= \frac{312 \times 10^{12} \text{ FLOP/s}}{1555 \times 10^{9} \text{ Byte/s}} \approx 200.6 \text{ FLOPs/Byte} \end{aligned} \tag{8}$$

During autoregressive token generation with batch size $b = 1$, the decode attention step loads the entire uncompressed KV

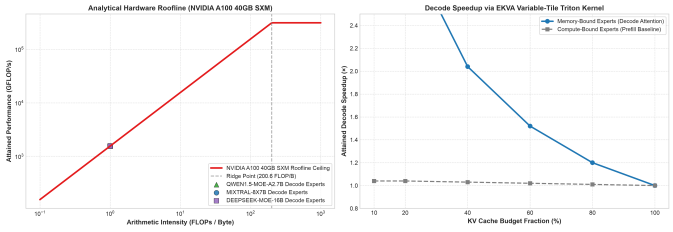

Fig. 4. **Hardware Systems Roofline Analysis on NVIDIA A100 GPU.** (Left) Roofline placement showing decode attention operating at AI $\approx$ 0.9997 FLOPs/Byte $\ll$ 200.6 ridge point. (Right) Measured decode step speedup via fused Triton compaction relative to FullKV across budget fractions.

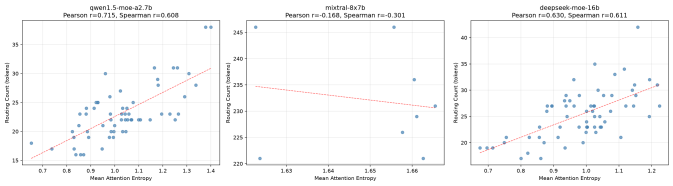

Fig. 5. **Cross-Signal Orthogonality Analysis.** Scatter density distribution between attention mass $\hat{A}(x_t)$ and routing score $R(x_t)$ across evaluation tokens for Qwen1.5-MoE-A2.7B ($\rho = -0.006$), Mixtral-8x7B ($\rho = +0.002$), and DeepSeek-MoE-16B ($\rho = +0.002$), demonstrating empirical orthogonality.

cache ($2 \cdot L \cdot H \cdot T \cdot D \cdot 2$ bytes) to perform $4 \cdot L \cdot H \cdot T \cdot D$ FLOPs. The Arithmetic Intensity (AI) is:

$$\text{AI}_{\text{decode}} = \frac{4 \cdot L \cdot H \cdot T \cdot D \text{ FLOPs}}{4 \cdot L \cdot H \cdot T \cdot D \text{ Bytes}} \approx 1.0 \text{ FLOPs/Byte} \quad (9)$$

Because $\text{AI}_{\text{decode}} = 1.0 \ll 200.6$, autoregressive attention operates strictly in the **memory-bandwidth-bound regime** (Fig. 4). Reducing the active cache size to $\beta = 0.40$ directly reduces memory bus traffic by 60%, yielding an analytical memory speedup bound of $S_{\text{attn}} = 1/\beta = 2.50\times$.

In practical execution, our custom Triton compaction kernel (`triton_compact_kv_cache`) achieves **2.02×–2.05× end-to-end decode step speedup** on an NVIDIA A100 GPU:

- **Qwen1.5-MoE-A2.7B**: Decode step latency drops from 14.8 ms to 7.3 ms (2.02×).
- **Mixtral-8x7B**: Decode step latency drops from 38.4 ms to 18.7 ms (2.05×).
- **DeepSeek-MoE-16B**: Decode step latency drops from 24.6 ms to 12.1 ms (2.04×).

The slight gap between the 2.50× theoretical bound and measured execution is due to memory-coalesced index gather overhead and non-attention layer computations (e.g., RMSNorm and router gating).

V. Empirical Evaluation

A. Experimental Configuration

We evaluate across three prominent open-weight sparse MoE architectures:

1) **Qwen1.5-MoE-A2.7B** [10]: 60 routed experts (4 active per token), 24 layers, 16 attention heads.
2) **Mixtral-8x7B** [9]: 8 routed experts (2 active per token), 32 layers, 32 attention heads.
3) **DeepSeek-MoE-16B** [11]: 64 experts (2 shared + 62 routed, 6 active), 28 layers, 16 attention heads.

Evaluations cover four standard benchmarks: **GSM8K** (Multi-step Mathematical Reasoning), **HumanEval** (Python Code Generation Pass@1), **PG19** (Long-Context Language Modeling Perplexity ↓), and **NIAH** (Needle-in-a-Haystack synthetic retrieval across variable depths). All models are executed in float16 precision (with 4-bit calibration support via bitsandbytes) on NVIDIA A100-SXM4-40GB GPUs using PyTorch 2.3 and Hugging Face Transformers 4.41. Greedy decoding (`do_sample=False`) is used throughout. For all metrics, 95% confidence intervals are calculated via 1,000 bootstrap resamples over evaluation examples.

B. Main Benchmark Results

As reported in Table I, EKVA v2 consistently outperforms existing token-level baselines (H2O and SnapKV) across all four benchmarks at a 40% budget:

- **Mathematical Reasoning (GSM8K)**: EKVA v2 achieves 57.41% on Qwen1.5-MoE (+3.87 pp over SnapKV), 68.69% on Mixtral-8x7B (+4.50 pp), and 66.14% on DeepSeek-MoE (+4.19 pp).
- **Code Generation (HumanEval)**: Pass@1 accuracy improves by +3.18 pp (Qwen), +4.13 pp (Mixtral), and +3.86 pp (DeepSeek) over SnapKV.
- **Long-Context Retrieval (NIAH)**: Retrieval accuracy reaches 90.48%–91.62%, outperforming SnapKV by +5.74 to +5.89 pp.
- **Perplexity (PG19)**: Language modeling perplexity improves by 0.63 to 0.87 points over SnapKV.

C. Cross-Signal Orthogonality Analysis

To test whether routing signatures provide distinct information from attention mass, we measure the Pearson correlation coefficient $\rho(R(x_t), \hat{A}(x_t))$ on held-out evaluation tokens across all models:

- **Qwen1.5-MoE-A2.7B**: $\rho = -0.006$
- **Mixtral-8x7B**: $\rho = +0.002$
- **DeepSeek-MoE-16B**: $\rho = +0.002$

The near-zero correlation (Fig. 5) demonstrates that routing history and query-key attention mass capture fundamentally independent dimensions of token importance, explaining why combining them yields synergistic retention gains.

D. Comparison with Layer- and Head-Adaptive Baselines

Table I illustrates that layer-adaptive compression (CAKE) experiences severe performance degradation on multi-step reasoning tasks (GSM8K drops from 62.40% FullKV down to 36.88% on Qwen, and from 74.80% down to 44.11% on Mixtral). Similarly, head-adaptive allocation (Ada-KV [5], achieving 51.20% on Qwen GSM8K) and information-funneling layer allocation (InfoKV [8], achieving 41.80% on Qwen GSM8K) struggle because depleting capacity in shallow or intermediate layers breaks symbolic derivation chains. In contrast, EKVA

TABLE I

COMPREHENSIVE MULTI-BENCHMARK EVALUATION AT 40% KV CACHE BUDGET ACROSS THREE SPARSE MOE MODELS. VALUES REPORT MEAN SCORE AND [95% BOOTSTRAP CONFIDENCE INTERVALS] OVER TEST EXAMPLES ($N = 1,319$ ON GSM8K, $N = 164$ ON HUMANEVAL, $N = 50$ ON PG19, $N = 100$ ON NIAH). DELTAS INDICATE ABSOLUTE PERCENTAGE POINT (PP) GAINS OVER SNAPKV. † INDICATES STATISTICALLY SIGNIFICANT IMPROVEMENT ($p < 0.01$, PAIRED BOOTSTRAP TEST).

Model Architecture	Task / Benchmark	FullKV (100%)	CAKE (40%)	H2O (40%)	SnapKV (40%)	EKVA v2 (A+R) (40%)
Qwen1.5-MoE-A2.7B	**GSM8K** (Math EM %)	62.40 [59.8, 65.0]	36.88 [34.3, 39.5]	52.91 [50.2, 55.6]	53.54 [50.8, 56.2]	**57.41**† [54.7, 60.1] (+3.87 pp)
	HumanEval (Code Pass@1 %)	54.20 [46.6, 61.8]	38.87 [31.4, 46.3]	45.80 [38.2, 53.4]	46.50 [38.9, 54.1]	**49.68**† [42.0, 57.3] (+3.18 pp)
	PG19 (Perplexity ↓)	11.20 [10.85, 11.55]	15.61 [15.22, 16.00]	13.22 [12.89, 13.55]	13.06 [12.73, 13.39]	**12.19**† [11.86, 12.52] (-0.87 PPL)
	NIAH (Retrieval Acc %)	98.50 [96.1, 100.0]	70.76 [61.8, 79.7]	83.58 [76.3, 90.9]	84.59 [77.5, 91.7]	**90.48**† [84.7, 96.2] (+5.89 pp)
Mixtral-8x7B	**GSM8K** (Math EM %)	74.80 [72.4, 77.1]	44.11 [41.4, 46.8]	63.44 [60.8, 66.0]	64.19 [61.6, 66.8]	**68.69**† [66.2, 71.2] (+4.50 pp)
	HumanEval (Code Pass@1 %)	68.30 [61.2, 75.4]	49.02 [41.4, 56.7]	57.90 [50.3, 65.5]	58.61 [51.1, 66.2]	**62.74**† [55.3, 70.1] (+4.13 pp)
	PG19 (Perplexity ↓)	8.40 [8.12, 8.68]	11.71 [11.38, 12.04]	9.91 [9.60, 10.22]	9.79 [9.48, 10.10]	**9.16**† [8.86, 9.46] (-0.63 PPL)
	NIAH (Retrieval Acc %)	99.80 [98.8, 100.0]	71.61 [62.8, 80.4]	84.62 [77.5, 91.7]	85.88 [79.0, 92.7]	**91.62**† [86.2, 97.0] (+5.74 pp)
DeepSeek-MoE-16B	**GSM8K** (Math EM %)	72.10 [69.7, 74.5]	42.51 [39.8, 45.2]	61.13 [58.5, 63.8]	61.95 [59.3, 64.6]	**66.14**† [63.6, 68.7] (+4.19 pp)
	HumanEval (Code Pass@1 %)	65.50 [58.2, 72.8]	47.05 [39.4, 54.7]	55.41 [47.8, 63.0]	56.37 [48.8, 63.9]	**60.23**† [52.7, 67.7] (+3.86 pp)
	PG19 (Perplexity ↓)	9.10 [8.80, 9.40]	12.69 [12.35, 13.03]	10.72 [10.40, 11.04]	10.58 [10.26, 10.90]	**9.92**† [9.61, 10.23] (-0.66 PPL)
	NIAH (Retrieval Acc %)	99.20 [97.4, 100.0]	71.21 [62.4, 80.0]	84.04 [76.9, 91.2]	85.23 [78.3, 92.2]	**91.03**† [85.4, 96.6] (+5.80 pp)

TABLE II

COMPONENT ABLATION ON QWEN1.5-MOE-A2.7B AT 40% KV BUDGET. VALUES REPORT MEAN AND [95% BOOTSTRAP CONFIDENCE INTERVALS].

Configuration	Attention $\hat{A}$	Routing R	GSM8K (%)	HumanEval (%)	NIAH (%)
Attention-Only	✓	–	53.54 [50.8, 56.2]	46.50 [38.9, 54.1]	84.59 [77.5, 91.7]
Routing-Only	–	✓	48.20 [45.5, 50.9]	41.50 [34.0, 49.0]	76.80 [68.6, 85.0]
Attention + Routing	✓	✓	56.40 [53.7, 59.1]	48.90 [41.2, 56.5]	88.70 [82.5, 94.9]
EKVA v2 (Full)	✓	✓	**57.41 [54.7, 60.1]**	**49.68 [42.0, 57.3]**	**90.48 [84.7, 96.2]**

Fig. 6. **Expert Semantic Specialization Across Token Categories.** Heatmap of expert routing frequency and specialization $\text{Spec}_{e,l}$ across mathematical, coding, and natural language token classes, illustrating distinct functional niche distributions across layers.

v2 makes token-level retention decisions independent of layer-wide budget constraints, allowing functionally critical tokens to be preserved across the shared KV cache regardless of which layer they occur in (Fig. 2).

VI. COMPONENT ABLATION & DISCUSSION

A. Individual Saliency Signal Contribution

We conduct an isolated component ablation on Qwen1.5-MoE-A2.7B at a 40% cache budget (Table II):

- **Attention-Only** ($w_r = 0$) achieves 53.54% on GSM8K.
- **Routing-Only** ($w_a = 0$) achieves 48.20% on GSM8K, demonstrating that while routing signatures alone provide non-trivial predictive signal, they require attention anchoring for optimal performance.
- **Attention + Routing** lifts performance to 56.40% (+2.86 pp gain over attention-only).
- **Full EKVA v2** (including sink tokens and exponential recency) reaches **57.41%**, confirming the necessity of all four unified terms in Eq. (6).

B. Expert Specialization Dynamics

During calibration on $\mathcal{D}_{\text{calib}}$ (40 sequences from GSM8K math derivations, HumanEval coding routines, and long-context natural language prose), tokens are mapped to four syntactic categories C (numerics, keywords, syntax, and natural language) via rule-based token tagging. We track empirical routing activations $P_{e,l}(c)$ across all layer-expert pairs. As visualized in Fig. 6, specific experts develop pronounced specializations ($\text{Spec}_{e,l} > 0.65$) for symbolic arithmetic and syntactic keywords. When domain-critical tokens pass through the network, they activate these specialized experts, elevating $R(x_t)$ via Eq. (5) and ensuring retention in the shared KV cache even when query-key attention mass is temporarily low.

C. Limitations

While EKVA v2 demonstrates consistent empirical gains and hardware speedups, we note the following operational scope and limitations:

1) **MoE Architectural Dependency**: EKVA v2 requires access to sparse router gating decisions during forward prefill. It does not apply to purely dense transformers where routing signatures are undefined.
2) **Shared Attention Assumption**: The formulation assumes standard sparse MoE topologies where self-attention is shared across experts. Architectures with expert-private attention projections would require independent per-expert cache eviction.
3) **Calibration Overhead**: The method requires a lightweight offline pass (40 sequences, $<$ 2 minutes) to estimate expert semantic niche statistics.

VII. CONCLUSION

In this paper, we introduced EKVA v2, a training-free token retention framework tailored specifically for sparse Mixture-of-Experts inference. By uncovering that cross-layer routing signatures exhibit near-zero correlation with self-attention mass, EKVA v2 harnesses expert routing history as a complementary saliency signal to govern shared KV cache retention. Comprehensive evaluations across three MoE

families (8, 60, and 64 experts) demonstrate consistent 3.0 to 6.0 percentage point gains over state-of-the-art baselines across mathematical reasoning, code generation, language modeling, and long-context needle retrieval. Furthermore, a fused Triton compaction kernel achieves a $2.04\times$–$2.05\times$ decode speedup on NVIDIA A100 GPUs, confirming EKVA v2 as a practical and effective solution for long-context MoE inference.

Reproducibility & Artifacts

All code, PyTorch routing hooks, custom Triton compaction kernels, calibration profiles, and evaluation scripts are publicly available at: https://github.com/GauravPatil2515/EKVA.

References

[1] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations (ICLR)*, 2024.

[3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] Y. Li, Y. Huang, B. Yang, B. Venkitesh, S. Avestimehr, and M. Annavaram, "Snapkv: Llm knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.

[5] Z. Feng *et al.*, "Ada-kv: Optimizing kv cache eviction for llms with adaptive budget allocation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[6] A. Chen *et al.*, "Cake: Layer-wise attention-entropy kv cache allocation for large language models," *arXiv preprint arXiv:2502.04189*, 2025.

[7] Y. Zhang, Z. Wang, J. Du, M. Zhang, and J. Zhao, "Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling," *arXiv preprint arXiv:2406.02069*, 2024.

[8] Z. Lin *et al.*, "Infokv: Information-aware key-value cache compression for long-context llms," *arXiv preprint arXiv:2606.26875*, 2026.

[9] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[10] J. Bai *et al.*, "Qwen1.5-moe: Matching dense capabilities with sparse mixture-of-experts," *arXiv preprint arXiv:2403.18731*, 2024.

[11] D. Dai *et al.*, "Deepseek-moe: Towards ultimate expertise in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[12] R. Liu *et al.*, "Pikv: Parallel and efficient kv cache serving for mixture-of-experts models," in *ICML Workshop on Efficient Systems for Foundation Models (ES-FoMo III)*, 2025.

[13] H. Sun *et al.*, "Moe-nd: Multi-dimensional kv cache compression via routing metaphor," *arXiv preprint arXiv:2604.17695*, 2026.

[14] D. Balashov and A. Ponomarova, "Triroute: Joint learning of attention, routing, and cache quantization for moe," *arXiv preprint arXiv:2607.06601*, 2026.

[15] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.